System Architecture Plan: HR & Access Control System

This document outlines the full-stack blueprint to implement the normalized database system. The architecture is divided into three distinct layers: Database, Backend API (Python), and Frontend Dashboard (TypeScript).

---

1. Database Layer (OLTP Relational Storage)

The database implements the **BCNF** schema designed during the normalization process.

Data Definition Language (DDL) Blueprint

Below is the relational structure mapping. All relationships maintain strict Referential Integrity using Cascaded or Restricted foreign keys.

```
-- 1. Independent Lookup Tables
CREATE TABLE SCHEDULES (
    id SERIAL PRIMARY KEY,
    shift_name VARCHAR(50) NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL
);

CREATE TABLE DEPARTMENTS (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- 2. Core Entity Table
CREATE TABLE EMPLOYEES (
    id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    salary DECIMAL(10, 2) NOT NULL,
    department_id INT NOT NULL REFERENCES DEPARTMENTS(id) ON DELETE RESTRICT,
    schedule_id INT NOT NULL REFERENCES SCHEDULES(id) ON DELETE RESTRICT
);

-- 3. Dependent Transactional / Historical Tables
CREATE TABLE SALARY_HISTORY (
    id SERIAL PRIMARY KEY,
    employee_id INT NOT NULL REFERENCES EMPLOYEES(id) ON DELETE CASCADE,
    new_salary DECIMAL(10, 2) NOT NULL,
    change_date DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE ACCESS_LOGS (
    id SERIAL PRIMARY KEY,
    employee_id INT NOT NULL REFERENCES EMPLOYEES(id) ON DELETE CASCADE,
    log_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    event_type VARCHAR(20) NOT NULL -- 'Check-In', 'Check-Out'
);

CREATE TABLE LEAVE_REQUESTS (
    id SERIAL PRIMARY KEY,
    employee_id INT NOT NULL REFERENCES EMPLOYEES(id) ON DELETE CASCADE,
    type VARCHAR(50) NOT NULL,     -- 'Medical', 'Vacation', 'Personal'
```

```
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Pending' -- 'Pending', 'Approved',
'Rejected'
);

CREATE TABLE PAYSLIPS (
    id SERIAL PRIMARY KEY,
    employee_id INT NOT NULL REFERENCES EMPLOYEES(id) ON DELETE CASCADE,
    period_month VARCHAR(20) NOT NULL,
    period_year INT NOT NULL,
    net_salary_paid DECIMAL(10, 2) NOT NULL
);
```

2. Backend API Layer (Python)

The backend acts as an intermediary, exposing secure RESTful endpoints, handling business logic validation, and mapping relational records to JSON payloads.

- **Recommended Stack: FastAPI** (High performance, native asynchronous support, automatic OpenAPI/Swagger documentation generation) + **SQLAlchemy** (ORM) or **SQLModel**.

Core API Endpoint Matrix

Method	Endpoint	Description	Payloads / Query Parameters
`GET`	`/api/v1/employees`	Lists all employees with pagination	Query: `?page=1&limit=20`
`POST`	`/api/v1/employees`	Registers a new employee	JSON: `{name, email, salary, department_id, schedule_id}`
`GET`	`/api/v1/employees/{id}`	Retrieves details of a specific employee	Path: `id`
`POST`	`/api/v1/access-logs`	Registers a swipe card event (Check-In/Out)	JSON: `{employee_id, event_type}`
`GET`	`/api/v1/employees/{id}/payslips`	Retrieves payslip execution history	Path: `id`
`POST`	`/api/v1/leave-requests`	Submits a new time-off application	JSON: `{employee_id, type, start_date, end_date}`
`PATCH`	`/api/v1/leave-requests/{id}`	Approves or rejects a leave request	JSON: `{status: "Approved" | "Rejected"}`

Key Logic Handlers to Implement:

1. **Salary Modification Hook:** Whenever a `PATCH /api/v1/employees/{id}` updates the `salary` column, an automatic background trigger or service function must insert a row into `SALARY_HISTORY` documenting the transition.
2. **Access Log Validation:** Prevent sequential double Check-Ins or double Check-Outs by validating the database's most recent `event_type` timestamp for that worker.

3. Frontend Dashboard Layer (TypeScript)

The user interface handles state visualization, manages user interactive flows, and enforces type-safety constraints to minimize rendering errors.

- **Implemented Stack: Next.js 16** (App Router) + **React 19** + **TypeScript** + **Tailwind CSS v4** (via PostCSS) + **Axios** + **TanStack React Query v5** + **lucide-react** (icons).

TypeScript Interface Contracts (`/types/index.ts`) — ✅ Implemented

To align with the backend database schemas, frontend entities map to strict TypeScript structures:

```typescript
export interface Schedule {
  id: number;
  shiftName: string;
  startTime: string; // HH:mm:ss format
  endTime: string;
}

export interface Department {
  id: number;
  name: string;
}

export interface Employee {
  id: number;
  name: string;
  email: string;
  salary: number;
  departmentId: number;
  scheduleId: number;
}

export interface LeaveRequest {
  id: number;
  employeeId: number;
  type: 'Medical' | 'Vacation' | 'Personal';
  startDate: string; // YYYY-MM-DD
  endDate: string;
  status: 'Pending' | 'Approved' | 'Rejected';
}

export interface AccessLog {
  id: number;
  employee_id: number;
  log_date: string;
  event_type: 'Check-In' | 'Check-Out';
}

export interface Payslip {
  id: number;
  employee_id: number;
  period_month: string;
  period_year: number;
  net_salary_paid: number;
}
```

Modular UI Architecture View

The UI client layout divides responsibilities across functional modules:

1. **Login Page (`/src/app/login/page.tsx`)** — ✅ Implemented

- Renders a centered email/password login form.
- Sends credentials as `FormData` to the backend OAuth2 endpoint and stores the returned JWT via `AuthContext.login()`.

2. **Dashboard Shell (`/src/app/dashboard/page.tsx`)** — ✅ Implemented
   - Sidebar navigation with role-conditional links (Employees visible only to `Admin` / `HR`), user info display, and logout button.
   - Welcome card showing user name and access level.
   - Wrapped in `<ProtectedRoute>` for authentication enforcement.
3. **Administration Panel Component (`/components/EmployeeTable.tsx`)** — ⏳ Pending
   - Will render the master list of all employees.
   - Should handle searching, active routing to detail view profiles, and access to modals for new personnel records.
4. **Time and Attendance Module (`/components/AccessLogMonitor.tsx`)** — ⏳ Pending
   - Should show real-time or historical entries/exits using visual indicator tokens (Green for Check-In, Amber for Check-Out).
5. **HR Operations Workflow (`/components/LeaveApprovals.tsx`)** — ⏳ Pending
   - Dashboard interface for administrative managers to dynamically alter pending `LeaveRequest` status indicators using single-click button bindings.